



DIAL

Decentralized Intelligence Access Layer

Beyond Centralized AI: Peer-to-Peer Discovery and Routing for Distributed LLM Services

Distributed Computing Laboratory (DCL), EPFL

Headed by Prof. Rachid Guerraoui

Supervised by Zicong Hong

Alexander Wohlfahrt

May 2026

Abstract

Today, many of the most capable language models are accessed through centralized platforms. This works well, but discovery, routing, pricing, and access often depend on a small number of providers. As personal assistants, local models, institutional models, and specialized agents become more plausible, a useful question is how distributed AI services could be discovered and used without relying on a central registry.

This project introduces DIAL, the Decentralized Intelligence Access Layer, as a prototype for decentralized LLM service discovery and routing. DIAL builds a peer-to-peer network where nodes can host or access language-model backends, advertise capabilities through a Kademlia DHT, exchange richer profiles, and route queries to suitable peers. The system also includes HTTP APIs, command-line tools, and DIAL Chat, a web interface for interacting with the network and observing the routing process.

The evaluation focuses on routing behavior rather than generated answer quality. Controlled local benchmarks test different network sizes, latency conditions, dropout rates, routing policies, and mixed-capability workloads. The results show that capability-aware decentralized routing is feasible in a local prototype: clean scenarios complete reliably, mixed-capability queries are routed effectively in clean conditions, and selected peers are often close to a quality oracle used for analysis. The goal, however, is not only to find the single strongest node for every query; a useful decentralized router should also avoid fragile traffic concentration. The main limitation is candidate visibility: as the network grows, or when latency and dropout are introduced, the entry node may evaluate too small a part of the active network and tail latency can become timeout-heavy. The report therefore treats DIAL as an access-layer foundation that could later support broader discovery, load-aware routing, richer recommendation search, trust, privacy, incentives, verification, and multi-agent collaboration.

Contents

1	Introduction	1
1.1	Project Thesis	2
1.2	Implemented Scope	3
2	Background and Design Motivation	3
2.1	Why Decentralize LLM Access?	3
2.2	Lessons from BitTorrent and Peer-to-Peer Systems	4
2.3	Where Blockchain Fits	4
2.4	The PPAI Angle: Matching Supply and Demand	4
2.5	Design Position	4
3	Related Work	5
3.1	Peer-to-Peer Discovery and Decentralized Serving	5
3.2	LLM Routing, User Needs, and Model Profiles	5
3.3	Agent Discovery and PPAI	5
3.4	Future Layers: Aggregation, Privacy, and Trust	6
4	System Architecture	6
4.1	Design Principles	6
4.2	Architecture and Communication Flow	7
4.3	Component Responsibilities	8
4.4	DIAL Chat	8
5	Capability Matching and Routing	8
5.1	Query Demand and Node Supply	8
5.2	Candidate Scoring	9
5.3	Routing Algorithm	11
5.4	Scope of the Current Router	12
6	Implementation	12
6.1	Repository Organization	12
6.2	Node Runtime and Startup	12
6.3	LLM Backends and Prompts	13
6.4	LLM-Based Capability Classification	14

6.5	Routing and Query Execution	14
6.6	HTTP API and DIAL Chat	14
6.7	Evaluation Harness	16
6.8	Implementation Limitations	16
7	Evaluation	17
7.1	Evaluation Goals	17
7.2	Clean and Latency Runs	18
7.3	Scalability and Candidate Visibility	18
7.4	Dropout	19
7.5	Mixed-Capability Queries	19
7.6	Policy Comparison	20
7.7	Evaluation Summary	21
8	Future Work	21
8.1	Multi-Node Query Execution	21
8.2	Aggregation and Answer Evaluation	22
8.3	Capability Profile Calibration	22
8.4	Reputation	22
8.5	Incentives and Pricing	23
8.6	Faster Failure Detection	23
8.7	Load-Aware Request Spreading	23
8.8	Discovery Coverage and Adaptive Recommendations	24
8.9	Privacy-Aware Routing and Trust Domains	24
8.10	Security and Identity	25
8.11	Richer Matching Algorithms	25
8.12	Beyond the Prototype	26
9	Conclusion	26
	References	27

1 Introduction

Most users experience modern AI through a simple interface: a prompt goes in, an answer comes back. Behind that simplicity, many of the most capable systems are still delivered through a centralized service model. The 2026 AI Index reports that industry produced more than 90% of notable frontier models in 2025, and it also highlights how AI infrastructure remains concentrated around large data centers and leading chip supply chains [1]. This model works well today, but it is not the only possible shape for AI infrastructure. A plausible next step includes not only remote frontier models, but also personal assistants, local models, institutional models, domain-specific fine-tunes, and small services optimized for narrow tasks. Recent work on collaborative inference, decentralized LLM serving overlays, and personalized edge LLM agents points toward this more diverse setting [2–4].

If this capacity remains isolated, most users still depend on centralized gateways. If it can be discovered, compared, routed to, and trusted, individuals and organizations could share useful AI services while keeping more control over where computation and data flow. The infrastructure question is therefore not only *which model is strongest?* It is also: *how should models be discovered, selected, trusted, and used without overloading the same few providers?* LLM routing and profiling work already treats model selection as a central problem [5–7].

This project, DIAL (Decentralized Intelligence Access Layer), studies one concrete piece of that question: decentralized LLM service discovery and routing. The core idea is to treat machines that host or access LLM backends as network nodes. Each node publishes a capability profile, discovers other peers through a peer-to-peer layer, and routes queries to nodes whose backend is likely to answer well. A user can connect to any node and let the network find a suitable executor without relying on a central model registry.

The prototype is intentionally practical. It focuses on a minimum useful foundation: nodes join a local peer-to-peer network, publish capability advertisements through a Kademlia DHT, exchange richer profiles, classify query needs, score local and remote candidates, and answer locally or forward the query. Around this layer, the project provides command-line tools, HTTP APIs, DIAL Chat, and an evaluation harness. The central runtime path is therefore not just “send a prompt to another machine”: the entry node interprets the prompt as capability demand, uses that demand to discover peers, compares full peer profiles, and lets the selected node’s own LLM backend produce the answer. DIAL is therefore both a network prototype and an application-facing prototype: the routing layer can be measured, and the same layer can be used through DIAL Chat by a real user.

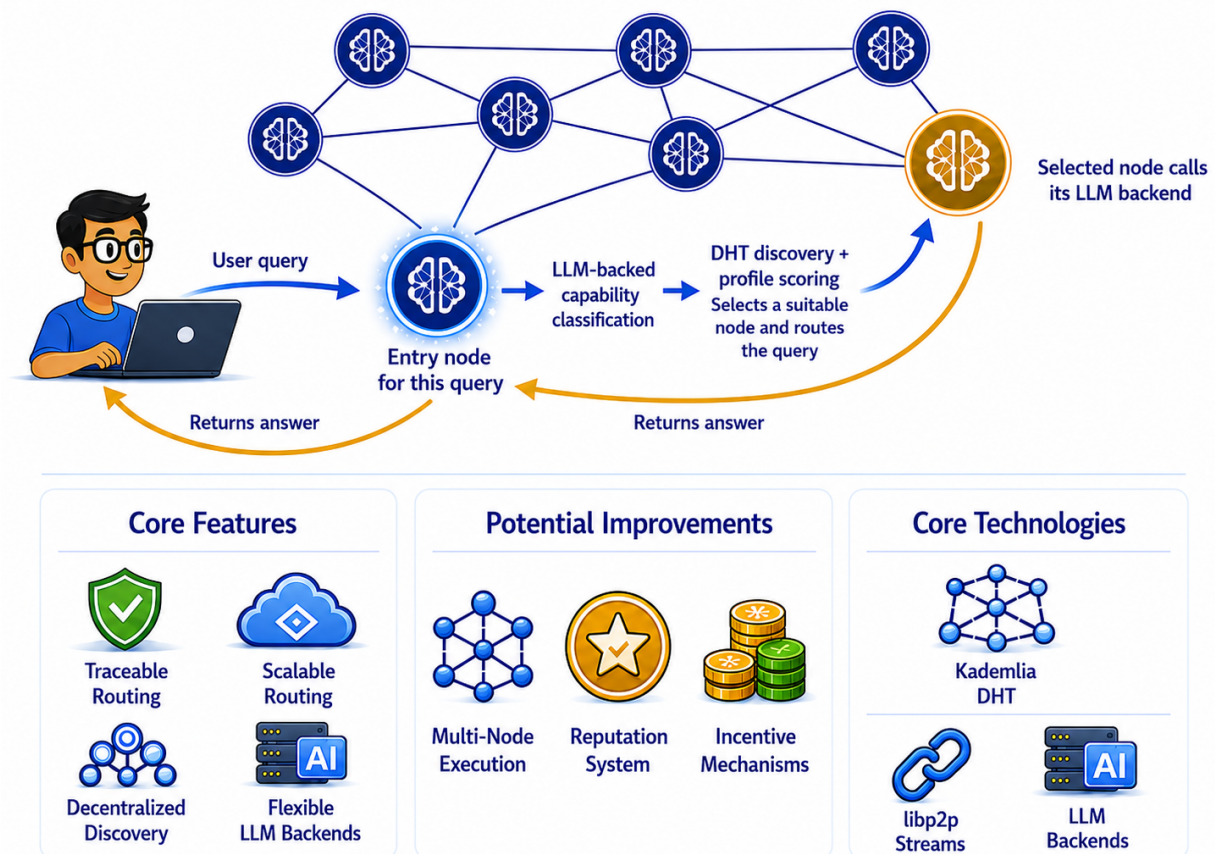


Figure 1: Overview of DIAL. A user query reaches an entry node for this request, which classifies capability needs, uses DHT discovery and profile scoring to select a suitable peer, routes the query, and returns the answer generated by the selected node’s LLM backend.

1.1 Project Thesis

The main thesis is the following:

A peer-to-peer network is a practical starting point for decentralized LLM service discovery and capability-aware query routing. DIAL focuses on the first operational layer of that idea: finding peers, exchanging profiles, and routing requests quickly. Blockchain was considered early in the project because the goal was decentralized access, but the implemented design uses the lighter P2P layer that better matches this first objective.

This is why the prototype uses libp2p and a Kademlia DHT rather than a blockchain-based design for the core network. Discovery is closer to BitTorrent and IPFS than to payment settlement: nodes need fast, changing information about who is available and what they can do [8–10]. Blockchain may become useful later if the network needs a shared record of payments, reputation, or other commitments between independent providers, but those features come with extra cost and complexity and are not needed for the discovery layer studied here [11, 12].

1.2 Implemented Scope

The project implements and evaluates the access layer of DIAL: decentralized discovery, profile exchange, candidate scoring, and query forwarding. In practice, this includes three main implementation outputs and one evaluation workflow.

1. A decentralized node architecture built around libp2p, a Kademlia DHT, profile exchange, capability scoring, and local or remote query execution.
2. A capability-aware routing model that supports both single-capability and weighted multi-capability requests. It classifies a query into a small capability mix, uses that mix for DHT discovery, and scores candidates against the full mix while leaving room for future load-aware policies.
3. An API layer and DIAL Chat web interface that make the network usable as an interactive application rather than only as a backend experiment.
4. An evaluation workflow used to measure whether the routing layer behaves correctly under controlled local conditions.

Stronger identity, privacy-preserving routing, abuse resistance, payments, and reputation are important extensions, but they belong to later layers of the system.

The implementation shows that a decentralized AI routing layer can be built with current tools, while exposing the next problems: broader candidate discovery, faster failed-peer handling, richer matching, answer aggregation, incentives, privacy, and reputation.

2 Background and Design Motivation

2.1 Why Decentralize LLM Access?

DIAL is motivated by a simple gap: as LLM services become more diverse, users still need a way to find and use the right one. A centralized platform can coordinate discovery, ranking, pricing, and access control, but it also becomes the place where those decisions are concentrated. This creates several tensions:

- **Provider concentration:** users often depend on a small number of large providers for availability, pricing, and access to strong models.
- **Specialization discovery:** niche model providers need a way to be found by clients that need their capabilities.
- **Privacy and locality:** some requests should stay near a user, institution, or trusted device.
- **Open participation:** a decentralized service layer should allow a new model provider to join without asking a central operator for permission.
- **Composable intelligence:** future queries may benefit from several models answering, checking, or debating rather than one monolithic model responding.

The project therefore focuses on one technical layer: service discovery and routing. The narrower question is whether peers can advertise what they can do, discover each other, compare profiles, and route a query in a measurable way.

2.2 Lessons from BitTorrent and Peer-to-Peer Systems

BitTorrent showed that a decentralized network can coordinate resource exchange without a central content server [9]. IPFS and libp2p generalize related ideas: content addressing, peer discovery, transport abstraction, and protocol negotiation [10, 13]. The useful analogy is not file sharing itself, but the coordination pattern: a user does not need to know in advance which machine will provide the resource.

Here, the resource is an LLM service exposed through a node profile. Peers advertise capabilities such as math, programming, writing, research, planning, or creative generation. An entry node uses those advertisements to find candidates and select an executor. The DHT acts as a decentralized capability index, while direct peer protocols retrieve fuller profiles, request recommendations, check liveness, and forward queries.

2.3 Where Blockchain Fits

Blockchain systems provide shared records that are difficult to modify and a common order for transactions [11]. These properties can be useful for payments, settlement, provenance, and reputation records. Blockchain-based AI trust frameworks also use smart contracts, trusted oracles, and decentralized storage to support auditability and trust [12]. However, putting every capability advertisement on-chain would impose costly global agreement on a problem that can tolerate temporary inconsistency.

DIAL therefore separates *discovery* from future *accounting and trust* layers. Discovery needs low-latency updates: if a peer leaves, the router can retry or choose another candidate. Payments, reputation, and provider commitments need stronger integrity and may later benefit from blockchain or other verifiable records. For the prototype, the core network is deliberately lighter: libp2p streams for direct communication and a Kademlia DHT for capability discovery.

2.4 The PPAI Angle: Matching Supply and Demand

PPAI helped shape the project vision because it describes a network where personalized edge LLM agents can be matched to user demand instead of leaving each user with only one local assistant [4]. The useful idea for DIAL is the supply-demand framing: users express demand through queries, while providers express supply through profiles, availability, quality, and possibly price or reputation.

DIAL narrows that broad idea to a runnable infrastructure layer. Supply becomes a `NodeProfile`; demand becomes a small capability vector produced by a classifier; and the DHT provides decentralized discovery. This is simpler than PPAI's full matching and delegation vision, but it gives that vision a concrete base: peers can be found, compared, selected, and measured before richer policies are added.

2.5 Design Position

This positions DIAL as an access-layer prototype for decentralized LLM services. The implemented focus is peer-to-peer discovery, profile exchange, and capability-aware routing. Layers such as trust, reputation, payments, privacy policies, and multi-node execution sit above that foundation. A DHT does not solve trust or economics, but the network first needs a way to find services before it can rank, verify, or reward them.

3 Related Work

Chapter 2 explained why the project uses a lightweight peer-to-peer discovery layer. This chapter has a different purpose: it positions DIAL relative to existing systems and papers. The main point is that DIAL combines two families of ideas that are often studied separately: peer-to-peer discovery and LLM routing.

3.1 Peer-to-Peer Discovery and Decentralized Serving

The networking part of DIAL follows the peer-to-peer tradition. Kademlia provides the structured DHT idea used to locate providers by key [8], while libp2p provides practical building blocks for peer identity, transports, protocols, and DHT participation [13]. BitTorrent and IPFS are useful background because they show that open networks can coordinate resource exchange and content retrieval without relying on a central content server [9, 10].

DIAL applies this pattern to LLM services rather than files or content blocks. Each peer is treated as a service with a profile and a model backend. This differs from systems such as Petals, which distribute parts of one large model across many machines [2]. It is closer in spirit to decentralized serving overlays such as PlanetServe, but DIAL studies a smaller question in a local prototype: whether peers can advertise capabilities, discover each other, and route queries measurably [3]. Work such as Lattica is also relevant because open AI networks eventually need robust connectivity across difficult network conditions, but that is outside the evaluated scope here [14].

3.2 LLM Routing, User Needs, and Model Profiles

The routing part of DIAL connects to work on choosing between models. FrugalGPT and RouteLLM show that model selection can trade off quality, cost, and latency instead of sending every prompt to the same model [5, 6]. Surveys of routing and cascading place these systems in a wider family of decision layers for LLM applications [15].

A related question is what a user request should be classified into before it can be routed. TUNA studies real human-AI conversations and organizes user needs into broad modes such as information seeking, synthesis, procedural guidance, content creation, social interaction, and meta-conversation [16]. Super-NaturalInstructions and HELM are useful from the task and evaluation side: they show that language-model behavior is commonly studied through families of tasks rather than one single capability [17, 18]. These works do not define DIAL’s exact eight labels, but they support the choice to use a small, interpretable set of common request families for a prototype router.

For DIAL, the most important lesson is that routing needs profiles. Universal Model Routing, RouteProfile, and vLLM Semantic Router all represent models or requests with richer signals than a model name alone [7, 19, 20]. DIAL uses a simpler capability vector because the goal is not to design the best semantic router. The goal is to make profile-based routing work across peers that are discovered dynamically through the network.

3.3 Agent Discovery and PPAI

PPAI is the closest conceptual neighbor because it describes personalized edge LLM agents that can be matched to user demand through query-agent scoring and load-aware delegation [4]. DIAL does

not implement the full PPAI matching algorithm. Instead, it implements the lower layer that such a system would need: peers join, publish capabilities, exchange profiles, route a query, and expose traces that can be inspected.

Recent agent-discovery work makes the same infrastructure problem explicit. Agents need to announce capabilities, be found, be ranked, and remain usable as peers join, leave, or fail [21]. Usable agent discovery also notes that node availability and service readiness are related but distinct problems [22]. For DIAL, this means discovery is only the first step: the router still needs profile and availability information to choose between candidates.

3.4 Future Layers: Aggregation, Privacy, and Trust

The current system selects one node to answer a query. The papers below are useful mainly because they show what could be layered on top of that first routing step. LLM-Blender and Mixture-of-Agents show that multiple model outputs can be ranked, fused, or layered to improve quality in multi-model settings [23, 24]. LLM-as-a-Verifier is relevant if future versions need a separate model to judge routed answers [25]. Privacy-preserving routing, verifiable inference, and blockchain-based AI trust work point to mechanisms that become relevant if the network moves beyond controlled local experiments and into open deployments [12, 26, 27].

These papers do not describe what DIAL already implements. They clarify the layers that would be needed after discovery and routing: answer aggregation, privacy policies, stronger identity, reputation, and verification.

4 System Architecture

4.1 Design Principles

The system was designed around four principles.

Every node is symmetric. There is no permanent gateway: any node can receive a query, route it, and answer if selected.

Discovery is decentralized. Nodes advertise capabilities through a Kademlia DHT instead of a central registry [8].

Profiles extend DHT hints. The DHT stores lightweight capability advertisements; full profiles are fetched directly for finer routing decisions.

The node answer backend is modular. A node can use Dummy for tests, Ollama or local Hugging Face Transformers for local inference, and EPFL AlaaS for hosted inference.

4.2 Architecture and Communication Flow

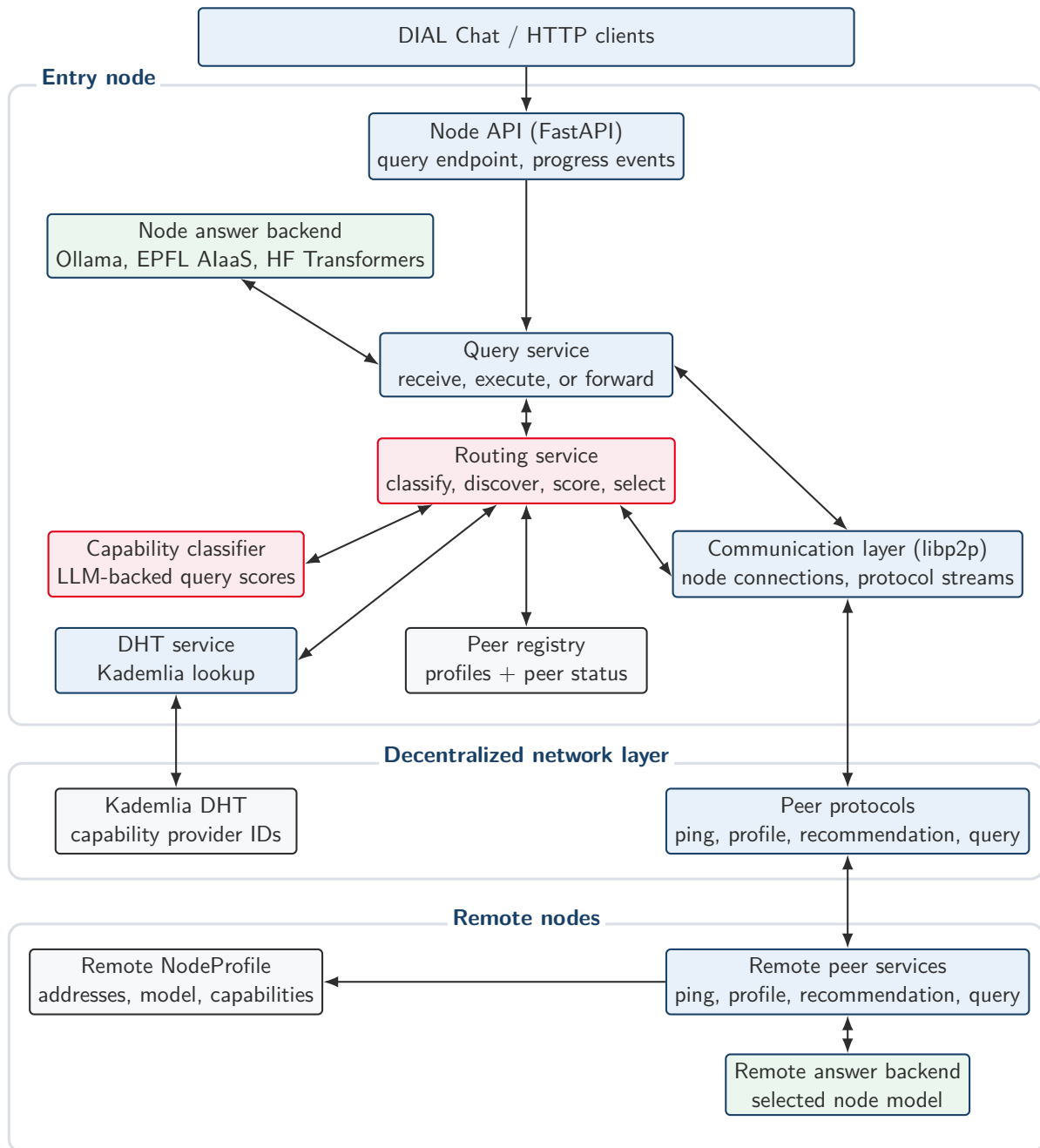


Figure 2: DIAL architecture and communication layers. The upper block shows the services inside the entry node for one request, the middle block shows the decentralized network layer, and the lower block shows other nodes exposing the same peer services and answer backend. Arrows indicate service calls or protocol exchanges.

Figure 2 shows the DIAL architecture and how its main blocks communicate. It is a component view, not the full query trace. DIAL Chat and HTTP clients submit requests through the node API. Some developer command line tools can also inspect or test the lower-level libp2p protocols directly. The query service owns the request lifecycle: it asks routing for a decision, then calls the node answer backend or forwards the query through libp2p.

The routing service owns the selection decision. It classifies the query, searches the Kademlia DHT for

capability providers, obtains profiles or recommendations through peer protocols, and uses the peer registry's profile and status information to score candidates. The distinction between entry node and remote node is only for one request: all nodes expose the same profile, recommendation, query, and answer backend services, so any node can take either role. Capability scoring at startup is a supporting mechanism, not part of the normal request path.

4.3 Component Responsibilities

The system is organized around four main responsibilities. This section describes what each part does; Chapter 5 explains the routing algorithm itself.

Client and API. DIAL Chat and HTTP clients submit requests to a node API. The API receives prompts and reports progress events, but it does not decide which node should answer. Separate command-line tools exist for local operation and debugging; some of them use libp2p directly.

Routing. The routing service chooses where a request should run. It uses the capability classifier, DHT discovery, peer profiles, recommendations, and peer-status information to compare candidates. The DHT only advertises a node's main capabilities; full profiles are fetched directly before scoring.

Peer-to-peer communication. The DHT provides coarse discovery by capability and returns provider peer IDs. Direct libp2p protocols handle the richer exchanges between nodes: pinging peers, fetching profiles, asking for recommendations, and forwarding queries. This direct path is enough for the current prototype. Gossipsub-style publish/subscribe is more useful when many peers need to receive the same event, such as profile-update notifications in a larger network [28].

Execution. Once a node is selected, the query service either calls the node's answer backend or forwards the request to the selected remote node. The answer is generated by the selected node's own backend, not by a central model service.

4.4 DIAL Chat

DIAL Chat is the user-facing web application on top of DIAL. Architecturally, it is a client of the node API: it lets a user write prompts, choose an available entry node, keep a conversation, and see progress events returned by that node. Those events expose the routing process in a readable way: classification, DHT lookup, candidate evaluation, recommendation, forwarding, and answer generation. This matters because DIAL is meant to be usable by normal applications, not only as a backend protocol. The implementation details of the web app are described in Chapter 6.

5 Capability Matching and Routing

5.1 Query Demand and Node Supply

The routing problem can be described as a matching problem between query demand and node supply. Let the set of supported capabilities be

$$C = \{\text{general, math, programming, writing, summarization, research, planning, creative}\}.$$

This set is deliberately compact, not a universal taxonomy of all LLM skills. It covers common

request families: information seeking, content processing or summarization, procedural help, writing or rewriting, coding, quantitative reasoning, and creative generation. These families are consistent with user-need, instruction-task, and evaluation work [16–18]. The small size also keeps DHT lookup and scoring simple.

A query is represented by a demand vector $q \in [0, 1]^{|C|}$, where $|C|$ is the number of supported capabilities. In normal operation, this vector is produced by a capability classifier. In the implementation, it is stored as a sparse map: omitted capabilities are treated as zero. The classifier uses the user request and returns one to three scored capabilities from the fixed set. For example, a simple math query may mostly need math, while a prompt such as “design a creative math puzzle and explain its solution” may need math, creative generation, and writing together.

Each node n also has capability scores s_n . The value $s_{n,c}$ means: how good node n is for capability c . The implemented prototype treats these scores as part of the node profile; in this prototype, they are configured or simulated rather than learned from a full model evaluation.

To compare a node with a query, the router computes a weighted average. For each capability, it multiplies how much the query needs that capability by how good the node is at it. Capabilities that matter more for the query therefore count more in the final score:

$$Q(n, q) = \frac{\sum_{c \in C} q_c s_{n,c}}{\sum_{c \in C} q_c}.$$

The division by $\sum_{c \in C} q_c$ is a normalization step. The classifier scores are treated as demand weights, not necessarily as percentages that already sum to 1. Dividing by the total demand keeps $Q(n, q)$ on the same 0–1 scale for simple and mixed queries. Without this normalization, a query with three requested capabilities could receive a larger score than a single-capability query only because more weights were added, not because the node is actually a better match.

For example, if a query receives need scores of 0.9 for math and 0.6 for writing, and a node scores 0.8 in math and 0.4 in writing, then

$$Q = \frac{0.9 \times 0.8 + 0.6 \times 0.4}{0.9 + 0.6} = 0.64.$$

The node is therefore a 0.64 capability match for that query. The same formula also works when the scores happen to sum to 1; in that case, the denominator is 1 and the expression reduces to the usual weighted sum.

This score is intentionally simple: interpretable, easy to inspect, and easy to extend. It also bridges to richer profile-based routing systems. PPAI-style query-agent matching, RouteProfile-style model profiles, and universal routing can be viewed as richer alternatives for the same interface [4, 7, 19].

The demand vector has two roles. First, its strongest capabilities guide DHT discovery: the router searches for peers advertising capabilities that look useful for the request. Second, the complete vector is used for scoring. A peer found through programming, for example, is still evaluated on the full request if the query also needs writing or planning.

5.2 Candidate Scoring

The prototype does not rank peers only by raw capability. Routing work shows that model selection trades off quality, cost, latency, and confidence [5, 6, 15]. Here, the “best” node is the node with the

best routing score: capability quality is combined with freshness, recent failures, and latency through a configurable policy.

$$R(n, q) = \alpha Q(n, q) + \beta F(n) - \gamma E(n) - \delta L(n).$$

Here $F(n)$ is a freshness score, $E(n)$ represents recent failures, and $L(n)$ represents latency. The weights define a policy. The default current policy uses a strong capability weight with equal smaller terms for freshness, failures, and latency:

$$R_{\text{current}}(n, q) = 0.70Q(n, q) + 0.10F(n) - 0.10E(n) - 0.10L(n).$$

Other implemented policies include `capability_only`, `latency_aware`, `reliability_aware`, and `balanced`. This makes the router a small experimental platform for studying matching trade-offs.

The goal is therefore not simply to find the absolute best capability score for every query. A node may have a strong model but still be a poor choice if it is slow, stale, or often unavailable. Conversely, a suitable local node can score well because it avoids network latency. This already helps the system avoid pure quality maximization, while future policies can add explicit load and request-spreading signals on the same scoring interface.

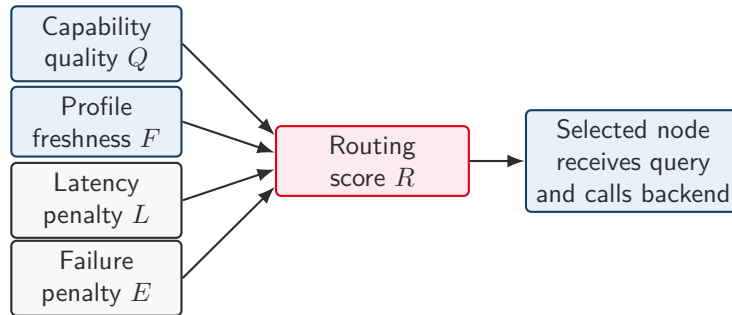


Figure 3: The implemented routing score is transparent by design: capability is the main signal, while freshness, failures, and latency make operational health visible before the selected node receives the query.

Table 1: Routing policies implemented by the prototype.

Policy	α	β	γ	δ	Intent
current	0.70	0.10	0.10	0.10	Prioritize capability while keeping freshness, failures, and latency visible.
capability_only	1.00	0.00	0.00	0.00	Measure pure quality matching without health signals.
latency_aware	0.60	0.05	0.05	0.30	Penalize slower peers more aggressively.
reliability_aware	0.60	0.05	0.30	0.05	Penalize recent failures more aggressively.
balanced	0.55	0.15	0.15	0.15	Combine capability with equal freshness, failure, and latency signals.

5.3 Routing Algorithm

Chapter 4 described the architecture components. This subsection describes the routing process in more detail: how those components are used by the implemented router for one query.

The router follows a staged process. In the implementation, the routing service selects a target, then the query service applies that decision by executing locally or forwarding the query to the selected peer.

1. Classify the user query into scored capability needs. This is the first LLM-backed step in a normal query: a classifier LLM, guided by a system prompt, maps the request to capability scores. For controlled evaluation, the required capabilities can also be injected directly. The implementation normalizes the result and keeps at most the top three capabilities.
2. Query the DHT for providers of the relevant capabilities. The router searches capabilities with demand at least 0.3; if all scores are below this discovery threshold, it still searches the strongest capability.
3. Fetch full profiles from the DHT providers and store or update them in the peer registry.
4. Build the pre-recommendation candidate set from the local node and available registry profiles that match at least one requested capability. In a fresh run, these profiles mostly come from the current DHT lookup; over time, the registry can also contain profiles learned earlier. All candidates are scored again for the current query.
5. If the best pre-recommendation candidate has a routing score of at least 0.65, select it as the target.
6. Otherwise, ask eligible remote candidates for recommendations. These recommendations expand the candidate set; recommended peers are then evaluated with the same routing score.
7. If no candidate reaches 0.65, fall back to the best candidate seen. If no candidate was found at all and the original query did not already include `general`, retry discovery once with the general capability. If this retry also finds nothing, return `no_suitable_node`.
8. Deliver the query to the selected target. If the target is the entry node itself, the query is executed locally. If the target is remote, the entry node opens a query protocol stream and sends the prompt, query identifier, and capability context to that peer.
9. Generate and return the answer. This is the second LLM-backed step in a normal query: the selected node calls its bound answer backend, returns the answer to the entry node if needed, and the entry node returns it to the user.

The 0.3 discovery threshold is used only to decide which capabilities should be searched in the DHT. It avoids querying the DHT for very weak capability hints, while the strongest capability is still searched if all scores are below 0.3. The 0.65 routing threshold is a practical cutoff: high enough to avoid accepting a weak pre-recommendation candidate immediately, but low enough to keep the router from asking for recommendations too often. These values were chosen empirically for this prototype and are exposed in the routing trace, together with whether the selected peer came from the first candidate pool, recommendations, or fallback selection.

Forwarding is intentionally not recursive. When a selected remote node receives a forwarded query, it answers with its own backend instead of running the whole routing process again. This avoids routing

loops and unbounded delegation. If forwarding fails, the entry node can exclude that peer and ask the router for another target, but forwarding attempts are capped.

5.4 Scope of the Current Router

The implemented router is intentionally a single-executor router. For one user request, it chooses one target node, sends the query there if needed, and returns one answer. It does not yet split a task across several nodes, aggregate several answers, use reputation, price, or privacy constraints, or optimize global network load. Those extensions fit the same matching interface, but they belong to the future-work layer rather than the current routing algorithm.

6 Implementation

6.1 Repository Organization

The prototype is implemented in Python. Runtime code lives under `src/`, command-line entry points under `src/cli/`, evaluation drivers under `benchmarks/`, scenario definitions under `benchmarks/scenarios/`, and DIAL Chat under `src/ui/`. The repository is organized around the main system responsibilities:

- `node.py` and `models.py` define the node runtime and shared data structures;
- the service modules implement discovery, profile exchange, recommendation requests, and query forwarding;
- `routing_service.py` and `routing_policy.py` implement capability matching, scoring, and node selection;
- `node_api_service.py` and `web_app.py` expose the node API and DIAL Chat;
- `benchmarks/run_benchmark.py` and `benchmarks/scenarios/` drive the local evaluation workloads.

Other files support this core path with environment loading, logging, backend adapters, health checks, protocol helpers, and small development utilities. The command-line tools and scripts are used to start nodes, launch local networks, inspect discovery behavior, and run diagnostic queries. The main user-facing interface is DIAL Chat, which lives under `src/ui/` and is described later in this chapter.

The public source code is available at <https://github.com/SpicyPaper/DIAL>.

6.2 Node Runtime and Startup

A single node can be started with `src/cli/run_node.py`, while a local network of several nodes can be launched with `scripts/start_network.sh`. Both read their configuration from the project `.env` file; full setup instructions are kept in the project `README`.

When a node starts, it creates a libp2p host, derives its peer identity, initializes its profile, connects to bootstrap peers, starts the DHT service, and advertises its top capabilities. The network script

launches several nodes with different ports, model names, and profiles. Runtime state is written under `.runtime/`.

The provided local setup advertises IPv6 loopback multiaddresses (`/ip6:::1/...`) because this was the local address mode used by the test machine when several nodes were run on one host. This is a deployment choice, not a routing assumption. libp2p multiaddresses can describe different network addresses and transport paths [29], so supporting IPv4, public IPv6, or relay-based addresses such as circuit relay paths [30] would mainly require extending the address-selection mode, launch configuration, and connectivity tests.

The concrete `NodeProfile` dataclass contains the fields needed by this runtime: `peer_id`, `reachable_addresses`, `model_name`, `advertised_capabilities`, `full_capability_scores`, `is_available`, and `timestamp_ms`. Architecturally, these are the profile data described in Chapter 4; here they become concrete fields used by discovery, caching, and routing.

Capabilities are set when the node starts. A node receives configured capabilities or role hints, and the local capability scorer turns them into a deterministic profile: usually one strong primary capability and weaker secondary capabilities. These scores are routing metadata, not measurements of the model's real skill, because the project focuses on discovery and routing rather than benchmarking the LLM itself.

6.3 LLM Backends and Prompts

The `LocalAgent` abstraction supports several model execution modes:

- **dummy**: deterministic placeholder responses for tests and controlled routing runs;
- **Ollama**: local inference through an Ollama server, recommended for demos;
- **AlaaS**: EPFL OpenAI-compatible inference API;
- **local Transformers**: a Hugging Face Transformers backend for fully local model loading.

This modularity matters because a decentralized AI network should not assume one runtime. A node may call a model from a local GPU, CPU runtime, institutional API, or specialized inference engine.

The local network is intentionally easy to run. Normal usage defaults to Ollama with the configured Qwen3 4B model [31]; controlled runs can also use a dummy backend when the goal is to exercise routing without generation variance.

In real runs, answer quality depends on the chosen backend model and on the system prompt used for the node. This is expected: prompting choices can change LLM behavior and robustness [32, 33]. The prompts used in this prototype are basic role prompts, not optimized prompts or a contribution of the project.

For practical reasons, the provided local network script normally launches several nodes against the same configured LLM backend. This is a resource-friendly demo setup, not an architectural restriction: each node process has its own peer identity, advertised profile, routing state, and system prompt, and custom launches can use different backend configurations. Backend calls are stateless unless conversation context is explicitly sent, so one node does not automatically remember another node's previous conversation.

6.4 LLM-Based Capability Classification

In normal interactive use, the entry node uses a classifier LLM to map the user prompt to capability scores. The supported capability set includes general, math, programming, writing, summarization, research, planning, and creative tasks. The classifier is guided by a dedicated system prompt and returns a small JSON object of scores, which makes classification an explicit routing step rather than an unstructured chat response.

The classifier output is not a final node choice. It is a compact description of what the query needs, and the router keeps at most the top three capabilities for discovery and scoring. Its quality depends on the selected classifier model and prompt, so controlled evaluation runs can bypass this LLM step by injecting `required_capabilities` directly. This lets Chapter 7 evaluate the network and routing behavior without also measuring classifier quality.

6.5 Routing and Query Execution

The routing implementation connects classification, discovery, and execution. For each query, `routing_service.py` normalizes the required capabilities, searches the DHT for peers advertising the relevant capabilities, fetches full profiles through the profile protocol, stores them in the peer registry, checks candidate liveness, and computes the routing score. The first candidate pool contains the local node and available matching registry profiles after this DHT refresh. If no candidate in that pool passes the threshold, the recommendation protocol asks eligible remote candidates for additional peer IDs. Those recommended peers are fetched and scored with the same routing logic.

Once a candidate is selected, `query_service.py` performs the execution step. If the selected candidate is the local node, the node calls its local agent directly. If the selected candidate is remote, the query is forwarded through the query protocol and the selected peer calls its own bound LLM backend. The response is then returned to the entry node and back to the client.

6.6 HTTP API and DIAL Chat

Each node exposes a small HTTP API through FastAPI [34]:

- `POST /api/query`: submit a query to the node;
- `GET /api/query/progress/<query_id>`: read routing progress events;
- `GET /api/profile`: inspect the node profile.

DIAL Chat reads available entry-node APIs from runtime configuration and presents a conversational interface. A user can choose an entry node, send a prompt, keep a conversation, and see how much recent context will be sent with the next request. The context meter makes clear that long conversations remain in the interface, while only the recent context window is included in the network request.

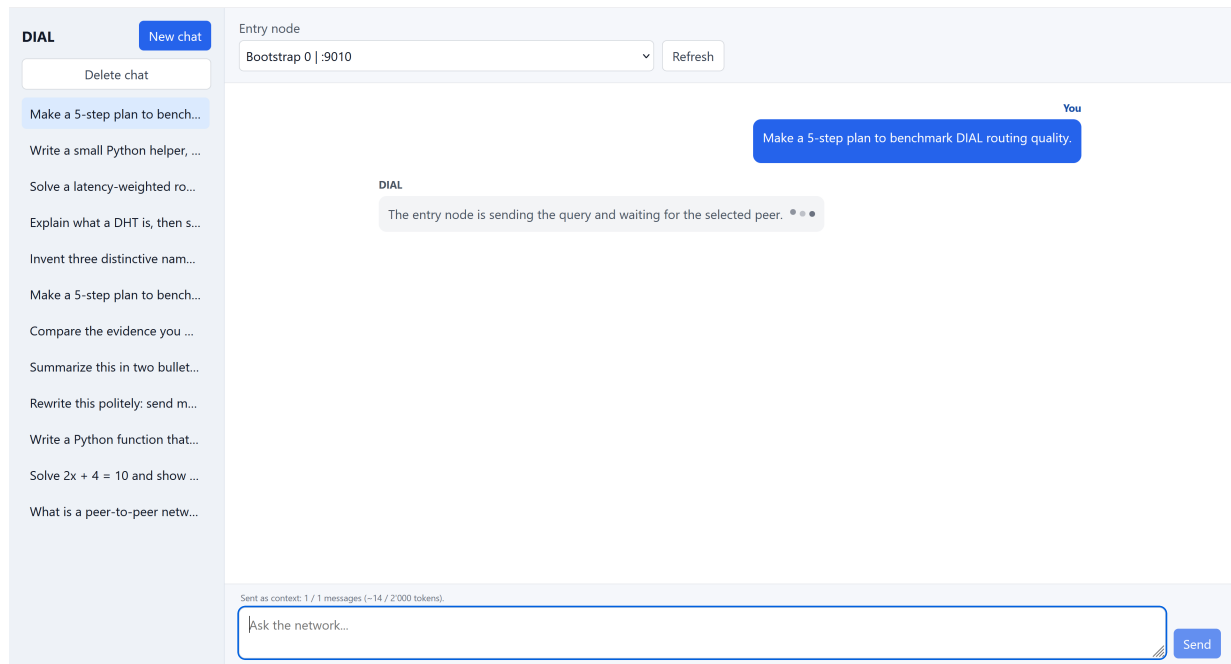


Figure 4: DIAL Chat main interface. The user selects an entry node, sends a query, and sees the recent conversation context that will be included in the next network request.

DIAL Chat also exposes routing progress events: classification, DHT lookup, candidate evaluation, recommendation, selection, forwarding, and answer generation. This makes the prototype easier to inspect because the user can see which node answered and why the routing path took time.

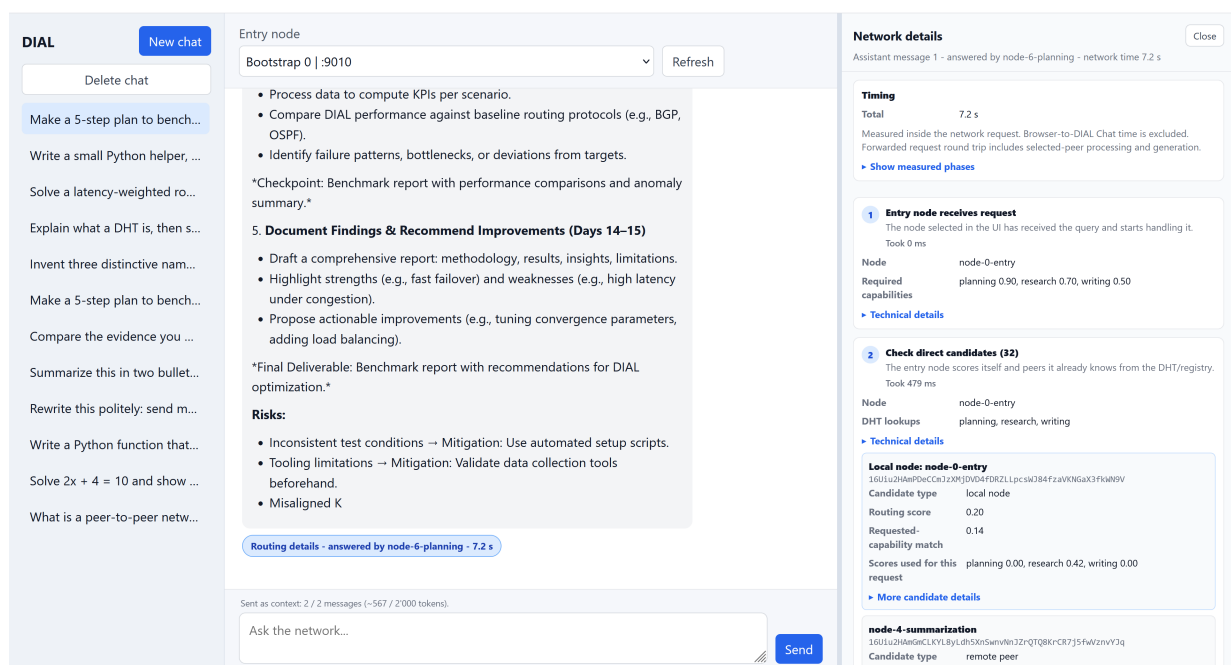


Figure 5: DIAL Chat network details panel. The interface shows timing information, required capabilities, candidate evaluation, and the selected peer that generated the answer.

6.7 Evaluation Harness

The evaluation runner starts local networks, injects scenario parameters, sends queries, records routing traces, and writes summaries. It can vary network size, latency, dropout, workload mix, and routing policy. Chapter 7 describes the actual scenarios and results.

The harness is designed to evaluate the network and routing layer, not the underlying LLMs. During benchmark runs, both the answer backend and classifier backend are set to `dummy`. Benchmark queries also provide `required_capabilities` directly in the API request, so the classifier is not called: the scenario defines the capability scores that the router should use. The selected node then returns a dummy answer instead of an LLM-generated one. This keeps the measured results focused on discovery, scoring, recommendation, forwarding, and failure handling. In normal interactive use, by contrast, the required capabilities come from the classifier LLM and the selected node calls its configured LLM backend.

6.8 Implementation Limitations

The current prototype is the result of a semester project, not a production network. Some mechanisms are therefore simplified deliberately so the project can focus on building and measuring the decentralized discovery and routing layer:

- capability scores are configured or simulated routing metadata, not measurements of real model skill;
- Gossipsub is implemented only as an optional profile-update notification path; normal local runs rely on DHT lookup and direct peer protocols;
- nodes have libp2p peer identities, but profiles are not signed and the prototype does not implement a trust mechanism for verifying advertised capabilities;
- unreachable peers are handled after failed connection or response attempts: the peer is marked unavailable, excluded, and routing is retried, but this availability information is not yet shared with the rest of the network;
- evaluation runs on a local machine with IPv6 loopback addresses and does not validate IPv4/public IPv6 addressing, NAT traversal, relay behavior, wide-area network behavior, or real provider heterogeneity;
- malicious behavior, false capability advertisements, and generated answer correctness are outside the evaluated scope;
- incentive, payment, and reputation systems are future work.

These limitations clarify the result rather than weaken it. As Chapter 7 shows, DIAL is a working prototype and gives encouraging routing results in the tested local scenarios. Several missing pieces, including stronger verification, reputation, incentives, privacy, and richer failure handling, are discussed as future improvements in Chapter 8.

7 Evaluation

7.1 Evaluation Goals

The benchmark suite evaluates the routing layer of DIAL under controlled local conditions. It does not evaluate natural-language answer quality. During these runs, capability requirements are provided directly by the benchmark and the selected node returns a dummy response. This keeps the measurement focused on the distributed system: discovery, profile fetching, scoring, recommendation, forwarding, and failure handling.

The goal is therefore not to prove that DIAL is ready for a public deployment. The results show whether the prototype can route requests correctly under controlled conditions, and where the current discovery and failure-handling logic starts to weaken. They do not validate real LLM answer quality, public network behavior, NAT traversal, provider trust, or privacy guarantees.

The latest benchmark snapshot contains 3,810 queries across 38 executed run folders. Overall, 90.4% of queries completed successfully, 63.6% selected the best active peer according to the pure capability-quality oracle, and 77.8% selected a top-3 peer. The average selected-to-oracle quality ratio was 81.8%. The failures were concrete: 276 queries ended with `no_suitable_node` and 91 ended with a timeout.

The main metrics are:

- **success rate**: fraction of queries returning a valid response;
- **top-1 accuracy**: whether the selected peer was the best active peer by pure capability quality;
- **top-3 accuracy**: whether the selected peer was among the three best active peers by pure capability quality;
- **quality ratio**: selected peer quality divided by oracle peer quality;
- **routing latency**: time spent in discovery, profile exchange, scoring, and forwarding decisions;
- **p95 end-to-end latency**: tail latency observed by the benchmark client;
- **candidate count**: how many peers were evaluated by the router;
- **candidate coverage**: evaluated candidates divided by active nodes.

For readability, rates and quality ratios are written as percentages in the text. The tables and plots keep the same quantities on a 0–1 scale.

Top-1 accuracy is deliberately strict. DIAL is not meant to send every request to the same highest-quality peer; it should choose a suitable peer while also considering reachability, latency, freshness, and eventually load. For this reason, quality ratio, top-3 accuracy, and candidate coverage are often more useful than top-1 alone.

One detail matters when reading the latency numbers. The benchmark runner uses a client-side timeout configured per scenario, while each node also has its own peer-query timeout. When a p95 value is close to 90, 120, 150, or 180 seconds, it usually means some requests waited until a benchmark timeout. These values should therefore be read as timeout pressure, not as normal response time for a successful route.

7.2 Clean and Latency Runs

The first result is positive: the basic routing path works. In clean 8-node runs, every query completed and the selected peer was almost always the oracle peer. Moderate simulated latency also remained stable. The 50–150 ms latency run still achieved 100% success and a 99.6% quality ratio.

Table 2: 8-node baseline and latency results.

Scenario	Queries	Success	Top-1	Top-3	Quality	P95 ms
No latency	120	1.000	0.967	1.000	0.991	277
50–150 ms latency	120	1.000	0.983	1.000	0.996	4,262
100–500 ms latency	120	0.917	0.867	0.917	0.899	120,023

High latency is different. At 100–500 ms simulated delay, success fell to 91.7%, and the p95 end-to-end latency reached the scenario's 120-second timeout range. This does not mean the score formula suddenly became bad, or that successful answers normally take that long. It means too many requests wait on slow network operations or failed forwards until the benchmark timeout is reached. Better timeout handling and more parallel discovery would matter more than small score-weight changes here.

In short, DIAL handles clean and moderate-latency routing well, but it should fail faster when network operations become very slow.

7.3 Scalability and Candidate Visibility

The scalability campaign is the most important stress test. It keeps latency at 50–150 ms and grows the network from 8 to 128 nodes. Success remains high, but quality degrades as the network grows: at 128 nodes, success is still 84.2%, but the quality ratio is 59.1% and top-1 accuracy is only 17.5%.

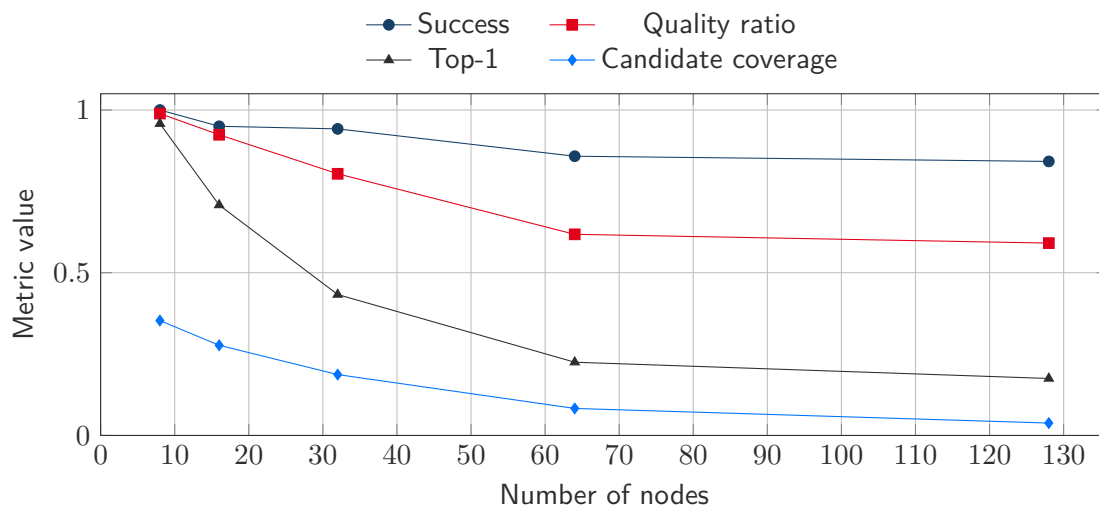


Figure 6: Scaling exposes the candidate-visibility bottleneck. The router still often completes the query, but it evaluates a shrinking share of the active network as node count grows.

The reason is visible in candidate coverage. At 8 nodes, the router evaluates about 35% of active

nodes. At 128 nodes, it evaluates only about 3.8%. Since the oracle is computed over all active profiles, larger networks create more opportunities for a better node to exist somewhere outside the evaluated set. This should not be read as a failure by itself. In a larger decentralized network, finding the exact top-1 or top-3 peer naturally becomes harder because there are more possible competitors, and DIAL should not concentrate all traffic on the same strongest nodes. The goal is to find a suitable peer and eventually spread load across the network. Still, the simultaneous drop in candidate coverage and quality ratio shows that discovery should improve for larger deployments. The implemented recommendation step already moves in this direction by letting peers suggest additional candidates, but it remains a conservative one-hop mechanism with a small per-peer limit. Further work should tune DHT/provider lookup behavior, return or refresh more useful providers per capability, improve recommendation diversity, and explore bounded multi-hop candidate expansion.

7.4 Dropout

The dropout campaign starts a 24-node network, kills a fraction of non-entry nodes, waits briefly, and then sends queries. Dropout reduces the candidate pool and increases `no_suitable_node`. The trend is not perfectly monotonic, because the removed nodes depend on the sampled dropout set, but the direction is clear: fewer active useful peers lead to weaker quality and long tail latency.

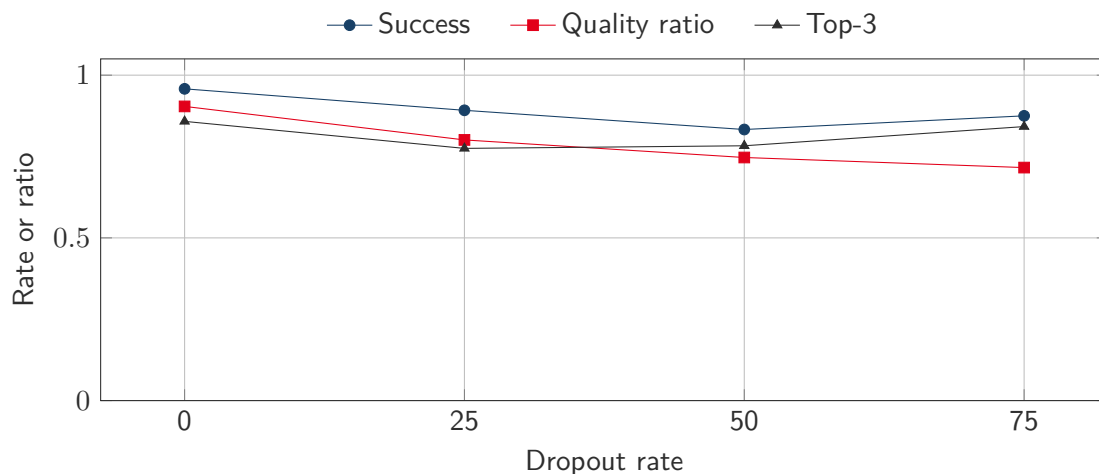


Figure 7: Dropout lowers routing quality and keeps tail latency near timeout-bound values. The 75% case has a small success rebound in this run, but its quality ratio and candidate count remain weak.

At 50% dropout, success is 83.3%, quality ratio is 74.7%, and p95 end-to-end latency is about 60 seconds. The average candidate count falls from 5.63 with no dropout to 2.71 at 50% dropout and 1.86 at 75% dropout. This is still encouraging: even after many peers disappear, the prototype usually returns an answer. The cost is weaker candidate visibility, which makes weaker selections more likely. Dropout should mainly be read as a discovery and liveness problem.

7.5 Mixed-Capability Queries

Mixed-capability queries are important because real prompts often require more than one skill. The clean 24-node mixed run is strong: all 45 queries complete, top-3 accuracy is 100%, and the quality ratio is 99.0%. This shows that the weighted capability model can work when the network is visible

and fast.

Table 3: Mixed-capability results under clean, latency, and dropout settings.

Scenario	Success	Top-1	Top-3	Quality	P95 ms
24 nodes, clean	1.000	0.956	1.000	0.990	2,325
16 nodes, 50–150 ms	0.756	0.600	0.622	0.687	90,029
16 nodes, 100–500 ms	0.711	0.489	0.533	0.618	120,026
24 nodes, 25% dropout	0.733	0.511	0.600	0.662	120,026

The same workload becomes much harder once latency or dropout is added. Mixed queries force the router to find peers that are good across several capability dimensions, so narrow candidate discovery hurts more than in single-capability queries. The practical improvement is to collect candidates per relevant capability, keep more than one provider per capability, and then score the combined requirement after deduplication. The rows with p95 values near 120 seconds should again be read as timeout-bound failures, not as normal response times for completed answers.

In short, the clean run validates the weighted capability representation. The stressed runs show that multi-capability routing needs broader candidate collection before the scoring formula can really help.

7.6 Policy Comparison

The policy runs compare five routing policies in four contexts: clean, high-latency, 50% dropout, and mixed-capability workloads. The winner changes by context, which is useful: it shows that policy design matters. But it also shows that policy tuning alone cannot fix low candidate coverage or long timeouts.

Table 4: Best policy by context in the latest benchmark snapshot.

Context	Best policy	Success	Top-1	Quality	P95 ms
Clean	latency_aware	1.000	0.842	0.948	1,365
High latency	capability_only	0.925	0.517	0.813	60,039
50% dropout	balanced	0.858	0.608	0.777	66,622
Mixed	reliability_aware	0.867	0.689	0.787	150,014

In clean runs, all policies reach 100% success and differences are small. Under high latency, `capability_only` performs best in this dataset, which is slightly counterintuitive but informative: adding latency penalties does not create better candidates, and under slow network operations it can still spend time evaluating or forwarding to poor paths. Under dropout, `balanced` does best, while mixed workloads benefit most from `reliability_aware`. These results should not be read as a final policy choice. They are better read as evidence that future policy studies should use more contrasted weights and scenarios designed to isolate one trade-off at a time. The large p95 value in the mixed row is also tied to the benchmark’s 150-second scenario timeout. The comparison is still useful: policy tuning can improve selection among visible peers, but it cannot compensate for candidates that were never discovered.

7.7 Evaluation Summary

The benchmark supports four main conclusions:

1. The end-to-end DIAL routing path works. Clean and moderate-latency results show that nodes can advertise capabilities, discover peers, select a suitable node, and return a response.
2. The main weakness is candidate visibility. As the network grows, the router evaluates a smaller share of active nodes, so the best peer is more likely to remain unseen.
3. Dropout and high latency mostly hurt through stale or slow paths. They increase failed routes, timeout-bound latency, and quality loss.
4. Mixed-capability routing works in clean conditions but needs broader candidate collection when latency, dropout, or larger networks are involved.

The report therefore focuses on the plots and tables that explain these points. The short version is that DIAL routes well when the network is small and visible; the next engineering work is broader discovery through DHT tuning and bounded recommendations, faster failed-peer handling, and more deliberate multi-capability candidate collection.

8 Future Work

DIAL currently implements the core path: nodes advertise capabilities, discover peers, route queries, and return answers through the selected node. Future work can improve this prototype in two ways. Some directions address limits observed during evaluation, such as candidate visibility, timeout-heavy failures, and policy comparison. Others extend the project toward a richer decentralized AI system, with several nodes working together, better trust signals, privacy constraints, and incentives for open participation. The ideas below are therefore both practical improvements to the current prototype and possible extensions of the broader DIAL vision.

8.1 Multi-Node Query Execution

The current router selects one node. A natural extension is multi-node execution, where a request can involve several specialized peers. One version is top- k execution: send the same query to several high-ranking nodes, then aggregate the answers. Another version is staged execution: route a request through a small sequence of nodes, for example planning first, then coding, then writing or verification. This could improve robustness and quality in several ways:

- compare answers and select the most consistent one;
- ask answer evaluator nodes to rate responses;
- combine specialized answers, for example one math model and one writing model;
- retry automatically when one selected peer times out;
- expose answer diversity to the user.

Multi-node execution fits the decentralized direction. A network of smaller specialized models may be more useful than one local model when their strengths are complementary. The trade-off is

coordination cost: more nodes can improve quality or robustness, but they also add latency and failure points. This connects to LLM-Blender and Mixture-of-Agents, which show that ranking, fusing, or layering multiple model outputs can improve quality [23, 24].

8.2 Aggregation and Answer Evaluation

Once several nodes answer, the system needs aggregation. Aggregator nodes could be regular DIAL nodes whose role is to fuse several responses into one final answer. They could select the strongest parts of each answer, remove contradictions, summarize long responses, or produce a final response with citations and confidence notes.

The network may also need answer evaluator nodes. Their role would be different from capability evaluators: instead of estimating whether a node is generally good at math or writing, they would judge whether a specific answer was useful, correct, complete, or well-grounded. This could happen during multi-node execution, or periodically as a quality check to see whether selected nodes are still producing good answers. Verifier-based systems such as LLM-as-a-Verifier are relevant because they turn answer checking into a reusable component [25].

This creates a new routing problem: the best set of nodes is not always the top k individually. The router may want diversity across capabilities, model families, latency classes, trust domains, and node roles. Matching then becomes a problem of choosing a good group of answer nodes, evaluator nodes, and possibly an aggregator node.

8.3 Capability Profile Calibration

The current prototype assigns capability profiles from configured or simulated scores. A stronger system should learn these profiles from evidence. Each node could periodically answer a small evaluation suite covering math, programming, writing, summarization, research, planning, creative tasks, and general questions. Existing work on user-need taxonomies and language-model evaluation could help design such a suite without reducing capability to one global score [16, 18]. The results could be graded by tests, human feedback, or verifier models, then used to update the node profile over time.

This could be handled by capability evaluator nodes: regular DIAL nodes whose task is to test other nodes and publish signed or local evaluation results. It would make advertised capabilities less dependent on manual role hints or self-declared profiles. It would also give reputation a better input: routing could learn not only whether a node is reachable, but whether it actually performs well on the tasks it claims to support.

8.4 Reputation

Reputation could measure whether a node is useful over time. Signals include uptime, latency, successful completion, user feedback, verifier scores, and consistency with other nodes. A first implementation can maintain local scores; later versions can publish signed reputation events or use verifiable logs.

Reputation is also a defense mechanism. In an open network, nodes may advertise capabilities they do not have. Routing should learn from observed behavior rather than trust advertisements blindly. Verifiable inference mechanisms such as VeriLLM point toward stronger versions, where selected execution claims can be checked publicly [27].

8.5 Incentives and Pricing

If the network becomes a market, providers need incentives. A future `NodeProfile` could include price metadata such as `price_per_query`, accepted payment methods, free quotas, or priority fees. Routing could then include cost in the matching objective, instead of choosing only by capability and operational health.

Blockchain or payment-channel mechanisms become relevant here. The discovery plane can remain peer-to-peer, while the payment layer records payments, escrows, staking, or rewards. This keeps the architecture close to BitTorrent's lesson: incentives help open networks stay robust, but should serve the resource exchange rather than replace discovery [9]. It also fits the broader PPAI direction, where matching requests to providers can include more than technical capability, such as cost and participation rules [4].

8.6 Faster Failure Detection

The dropout and high-latency benchmarks show that failure handling still needs work. The prototype often still returns an answer when many peers disappear, which is a good sign, but weaker candidate visibility and slow failed paths reduce quality and increase tail latency. Future versions could add:

- shorter adaptive timeouts based on recent latency;
- profile expiration and explicit unavailability propagation;
- background health checks;
- multiple candidate reservations before forwarding;
- fallback to local execution when remote confidence is low.

These changes would likely improve quality and p95 latency when peers are slow, stale, or unavailable. They also align with agent-discovery work that treats service readiness and changing peer availability as separate but interacting discovery problems [21, 22].

8.7 Load-Aware Request Spreading

The current policy framework can compare capability quality, freshness, failures, and latency, but it does not yet include an explicit load-balancing term. In a larger network, this matters because the best node on paper should not necessarily receive every request. If too many users choose the same strong peer, that peer may become slower or unavailable, while other good-enough peers remain unused.

A future router could therefore track simple load signals, such as recent selections, active requests, queue length, or provider-declared capacity. The router would still prefer capable nodes, but it could spread requests across several suitable peers instead of always selecting the highest score. This fits the PPAI idea of matching requests to providers under several constraints: quality matters, but so do availability, cost, trust, and network health [4]. Existing routing systems such as RouteLLM and the vLLM Semantic Router also support this view of routing as a policy decision over multiple signals, even if they do not target the same peer-to-peer setting [6, 20].

8.8 Discovery Coverage and Adaptive Recommendations

The evaluation suggests that several weaknesses come from limited candidate visibility. A first improvement direction is therefore to tune the discovery layer itself: DHT provider lookup breadth, provider refresh timing, profile refresh behavior, bootstrap warmup, and how many providers are kept per capability. These are engineering parameters rather than new research ideas, but they matter because the router can only score candidates it has discovered.

The current recommendation mechanism is also conservative. The entry node refreshes its candidate pool through the DHT, asks a few eligible peers for recommendations when needed, and stops. This keeps routing predictable, but it also makes discovery shallow. In a larger or uneven network, useful nodes may be beyond the peers initially found by the DHT or known by the entry node. This matters especially for mixed-capability queries, where the scoring formula needs enough candidates across several capabilities before it can make a good trade-off.

A future router could use adaptive multi-hop recommendation expansion. When candidate quality is low, the score gap is small, or the entry node has too few good options, it could ask recommended peers for their own recommendations. The goal would be to discover further into the network without turning routing into an uncontrolled search. The expansion should remain bounded:

- set a maximum hop depth, for example two or three recommendation hops;
- set a total recommendation budget per query;
- stop early when a reachable candidate passes the routing threshold;
- stop when new hops return mostly duplicates or already failed peers;
- penalize longer-path candidates unless their profile is clearly better;
- cache useful paths briefly, but expire them aggressively when peers change or disappear.

This trades extra discovery cost for a wider view of the network when one-hop routing is not enough. It is a middle ground between shallow DHT lookup and uncontrolled flooding: discovery can expand when useful, but still remains bounded.

8.9 Privacy-Aware Routing and Trust Domains

Not all queries should be routed to any peer. Some prompts contain private, institutional, or regulated information. The matching model can support trust constraints: route only to local peers, known identities, institutional peers, or peers with a privacy policy. This makes privacy a routing decision: some tasks may stay inside a trusted boundary, while less sensitive tasks may use a broader public network.

Privacy is harder in a public peer-to-peer network. DIAL should not assume that a raw prompt remains private once it is sent to an unknown execution node. The current design limits exposure during discovery: DHT lookup and recommendations exchange capabilities, profiles, and peer IDs, while the prompt is sent only to the selected node for execution. Still, the entry node and selected node can see the prompt.

This also opens the possibility of private or hybrid DIAL networks. An individual, lab, or organization could run a small private network of local specialized nodes instead of relying on one large centralized model. Such a network could keep sensitive prompts inside a trusted boundary while still benefiting

from specialization and aggregation. A hybrid version could route private tasks locally and use a broader public network only when policy allows it. This may improve privacy, specialization, and robustness, but it would need careful policies to avoid unnecessary latency or leakage. This is consistent with distributed inference systems such as Petals, where shared compute is useful but privacy remains an important deployment concern [2].

A future version could make this explicit with privacy modes such as `local-only`, `trusted-only`, `institutional`, or `public`. It could also reduce exposure by classifying locally before discovery, routing on capability metadata, redacting sensitive fields when possible, or sending only non-sensitive sub-tasks to public peers. More advanced approaches, such as privacy-preserving routing or privacy-preserving decentralized serving, are explored by PPRoute and PlanetServe [3, 26]. These approaches show that privacy can be a design direction for decentralized LLM routing, but it is not automatic and usually adds protocol or infrastructure cost.

8.10 Security and Identity

A future open network would need stronger identity guarantees than the current local prototype. Today, a node can publish a profile saying which capabilities it offers, but the prototype does not verify that this profile was signed by a stable identity or that the same actor is not creating many disposable nodes. This matters because reputation, payment, and capability evaluation only work if the network can attach past behavior to a persistent node identity.

A stronger design could require signed profiles, timestamps or freshness checks, and a clearer binding between a peer ID, public key, and advertised profile. It could also make fake or duplicated node identities harder to use, for example by linking reputation to longer-lived identities or requiring some form of verification before a node is trusted for sensitive or paid work. The lightweight discovery layer should therefore be paired with stronger authentication beyond local experiments. Lattica’s cross-NAT focus is relevant because an open network cannot assume that all peers are directly reachable [14].

8.11 Richer Matching Algorithms

The current vector model is deliberately simple. A later router could replace the current score with a broader utility objective:

$$U(n, q) = w_q Q(n, q) + w_r \text{Rep}(n) - w_p \text{Price}(n) - w_l \text{Latency}(n) - w_\rho \text{Risk}(n).$$

This would let the system choose, for example, a cheaper node above a quality threshold, a high-reputation node for sensitive requests, or a diverse group of nodes when aggregation is useful. Future matching can also use:

- semantic embeddings of queries and model descriptions;
- learned routing from historical outcomes;
- load-aware and energy-aware scheduling;
- price-quality trade-off curves;
- task decomposition across multiple specialist nodes;
- user-specific preferences for speed, quality, privacy, or cost.

This is where the project connects most strongly to PPAI. The network does not need to hard-code one matching algorithm; it can expose data and interfaces for competing policies. PPAI, UniRoute, RouteProfile, and vLLM Semantic Router suggest richer ways to represent tasks, models, policies, and constraints [4, 7, 19, 20].

8.12 Beyond the Prototype

If DIAL were extended beyond the current prototype, the natural order would be to make the network more reliable, add trust and reputation so routing can learn from experience, and then make participation more attractive for people or organizations that run useful nodes. Incentives for providers could mean visibility for reliable nodes, priority rules, credits, payments, or other rewards for sharing compute and specialized models. A later system would look less like a single application and more like an open protocol for AI service exchange.

9 Conclusion

This project built a working prototype for decentralized LLM service discovery and routing. Each node can host or access a language model, advertise capabilities, discover peers through libp2p and a Kademlia DHT, classify queries, rank candidates, and route execution. The system also includes HTTP APIs, DIAL Chat, and a benchmark suite for latency, scale, dropout, and policy weights.

The main result is that a peer-to-peer layer is a credible starting point for decentralized AI services when its role is clear. In local benchmarks, clean and moderate-latency networks route reliably, mixed-capability queries work well in clean conditions, and the capability vector model gives a path toward richer matching. The evaluation also clarifies that the aim is not to route every request to the same strongest node, but to choose suitable peers while keeping the network robust. The hard part is clear: candidate visibility does not yet scale well enough, and dropout, stale availability information, and timeout-heavy failure detection remain important bottlenecks. This points first to DHT/provider-discovery tuning, faster failed-peer handling, and bounded recommendation expansion. Load-aware routing remains a natural next direction once discovery is stronger.

The broader vision is an open AI service network where providers can join, advertise specialized models, build reputation, and be selected by transparent matching policies. Work on LLM routing, agent discovery, PPAI-style delegation, verifier models, and privacy-preserving routing suggests that this is becoming an active research direction. Blockchain is not needed for first-layer discovery, but may help later with payment settlement and verifiable reputation. In that sense, DIAL is best understood as a lightweight P2P foundation for discovery and routing today, with a path toward incentives, trust, and multi-model collaboration later.

References

- [1] Stanford Institute for Human-Centered Artificial Intelligence. *The 2026 AI Index Report*. Tech. rep. Stanford University, 2026. URL: <https://hai.stanford.edu/ai-index/2026-ai-index-report>.
- [2] Alexander Borzunov et al. *Petals: Collaborative Inference and Fine-tuning of Large Models*. 2023. arXiv: 2209.01188 [cs.LG]. URL: <https://arxiv.org/abs/2209.01188>.
- [3] Fei Fang et al. “PlanetServe: A Decentralized, Scalable, and Privacy-Preserving Overlay for Democratizing Large Language Model Serving”. In: *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*. USENIX Association, 2026, pp. 2111–2129. URL: <https://www.usenix.org/conference/nsdi26/presentation/fang>.
- [4] Zile Wang et al. *PPAI: Enabling Personalized LLM Agent Interoperability for Collaborative Edge Intelligence*. 2026. arXiv: 2605.18067 [cs.CL]. URL: <https://arxiv.org/abs/2605.18067>.
- [5] Lingjiao Chen, Matei Zaharia, and James Zou. *FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance*. 2023. arXiv: 2305.05176 [cs.LG]. URL: <https://arxiv.org/abs/2305.05176>.
- [6] Isaac Ong et al. *RouteLLM: Learning to Route LLMs with Preference Data*. 2025. arXiv: 2406.18665 [cs.LG]. URL: <https://arxiv.org/abs/2406.18665>.
- [7] Jingjun Xu et al. *RouteProfile: Elucidating the Design Space of LLM Profiles for Routing*. 2026. arXiv: 2605.00180 [cs.NI]. URL: <https://arxiv.org/abs/2605.00180>.
- [8] Petar Maymounkov and David Mazières. “Kademlia: A Peer-to-Peer Information System Based on the XOR Metric”. In: *Peer-to-Peer Systems*. Springer, 2002, pp. 53–65. DOI: 10.1007/3-540-45748-8_5.
- [9] Bram Cohen. *Incentives Build Robustness in BitTorrent*. 2003. URL: <https://www.bittorrent.org/bittorrentecon.pdf>.
- [10] Juan Benet. *IPFS - Content Addressed, Versioned, P2P File System*. Tech. rep. Protocol Labs, 2014. URL: <https://arxiv.org/abs/1407.3561>.
- [11] Satoshi Nakamoto. *Bitcoin: A Peer-to-Peer Electronic Cash System*. 2008. URL: <https://bitcoin.org/bitcoin.pdf>.
- [12] Mohamed Nassar et al. “Blockchain for Explainable and Trustworthy Artificial Intelligence”. In: *WIRES Data Mining and Knowledge Discovery* 10.1 (2020), e1340. DOI: 10.1002/widm.1340. URL: <https://doi.org/10.1002/widm.1340>.
- [13] Protocol Labs. *libp2p Documentation*. 2026. URL: <https://libp2p.io/docs/>.
- [14] Ween Yang et al. *Lattica: A Decentralized Cross-NAT Communication Framework for Scalable AI Inference and Training*. 2025. arXiv: 2510.00183 [cs.DC]. URL: <https://arxiv.org/abs/2510.00183>.
- [15] Yasmin Moslem and John D. Kelleher. *Dynamic Model Routing and Cascading for Efficient LLM Inference: A Survey*. 2026. arXiv: 2603.04445 [cs.NI]. URL: <https://arxiv.org/abs/2603.04445>.
- [16] Renee Shelby, Fernando Diaz, and Vinodkumar Prabhakaran. *Taxonomy of User Needs and Actions*. 2025. arXiv: 2510.06124 [cs.HC]. URL: <https://arxiv.org/abs/2510.06124>.
- [17] Yizhong Wang et al. “Super-NaturalInstructions: Generalization via Declarative Instructions on 1600+ NLP Tasks”. In: *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2022, pp. 5085–5109. DOI: 10.18653/v1/2022.emnlp-main.340. URL: <https://aclanthology.org/2022.emnlp-main.340/>.

- [18] Percy Liang et al. *Holistic Evaluation of Language Models*. 2022. arXiv: 2211.09110 [cs.CL]. URL: <https://arxiv.org/abs/2211.09110>.
- [19] Wittawat Jitkrittum et al. *Universal Model Routing for Efficient LLM Inference*. 2025. arXiv: 2502.08773 [cs.CL]. URL: <https://arxiv.org/abs/2502.08773>.
- [20] Xunzhuo Liu et al. *vLLM Semantic Router: Signal Driven Decision Routing for Mixture-of-Modality Models*. 2026. arXiv: 2603.04444 [cs.NI]. URL: <https://arxiv.org/abs/2603.04444>.
- [21] Shaolong Guo et al. "Agent Discovery in Internet of Agents: Challenges and Solutions". In: *IEEE Network* (2026). DOI: 10.1109/MNET.2026.3663837. arXiv: 2511.19113 [cs.NI]. URL: <https://arxiv.org/abs/2511.19113>.
- [22] Patrizio Dazzi et al. *Usable Agent Discovery for Decentralized AI Systems*. 2026. arXiv: 2604.23080 [cs.MA]. URL: <https://arxiv.org/abs/2604.23080>.
- [23] Dongfu Jiang, Xiang Ren, and Bill Yuchen Lin. "LLM-Blender: Ensembling Large Language Models with Pairwise Ranking and Generative Fusion". In: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Toronto, Canada: Association for Computational Linguistics, 2023, pp. 14165–14178. DOI: 10.18653/v1/2023.acl-long.792. URL: <https://aclanthology.org/2023.acl-long.792/>.
- [24] Junlin Wang et al. *Mixture-of-Agents Enhances Large Language Model Capabilities*. 2024. arXiv: 2406.04692 [cs.CL]. URL: <https://arxiv.org/abs/2406.04692>.
- [25] Jacky Kwok et al. *LLM-as-a-Verifier: A General-Purpose Verification Framework*. 2026. URL: <https://llm-as-a-verifier.notion.site/>.
- [26] Xidong Wu et al. *Privacy-Preserving LLMs Routing*. 2026. arXiv: 2604.15728 [cs.CR]. URL: <https://arxiv.org/abs/2604.15728>.
- [27] Ke Wang et al. *VeriLLM: A Lightweight Framework for Publicly Verifiable Decentralized Inference*. 2026. arXiv: 2509.24257 [cs.CR]. URL: <https://arxiv.org/abs/2509.24257>.
- [28] Protocol Labs. *What is Publish/Subscribe*. 2026. URL: <https://libp2p.io/docs/pubsub/>.
- [29] Protocol Labs. *Addressing*. 2026. URL: <https://libp2p.io/docs/addressing/>.
- [30] Protocol Labs. *Circuit Relay*. 2026. URL: <https://libp2p.io/docs/circuit-relay/>.
- [31] Ollama. *Ollama*. 2026. URL: <https://ollama.com/>.
- [32] Sander Schulhoff et al. *The Prompt Report: A Systematic Survey of Prompt Engineering Techniques*. 2025. arXiv: 2406.06608 [cs.CL]. URL: <https://arxiv.org/abs/2406.06608>.
- [33] Kaijie Zhu et al. *PromptRobust: Towards Evaluating the Robustness of Large Language Models on Adversarial Prompts*. 2024. arXiv: 2306.04528 [cs.CL]. URL: <https://arxiv.org/abs/2306.04528>.
- [34] FastAPI. *FastAPI Documentation*. 2026. URL: <https://fastapi.tiangolo.com/>.